Design and Development of Real-time Chat Application

Project Report Submitted in Partial Fulfilment of the Requirement for the Award of the Degree of

BACHELOR OF COMPUTER APPLICATION

By :

Syed Awaiz

Reg. No: U03EH23S0082

Under the guidance of

Dr. Sunetra Chatterjee

Assistant professor

IFIM COLLEGE (AUTONOMOUS)

8P & 9P, KIADB Industrial Area,

Electronics City Phase I, Bengaluru - 560100

Design and Development of Real-time Chat Application

Project Report Submitted in Partial Fulfilment of the Requirement for the Award of the Degree of

BACHELOR OF COMPUTER APPLICATION

By :

Syed Awaiz

Reg. No: U03EH23S0082

Under the guidance of

Dr. Sunetra Chatterjee

Assistant professor

IFIM COLLEGE (AUTONOMOUS)

8P & 9P, KIADB Industrial Area,

Electronics City Phase I, Bengaluru - 560100

CERTIFICATE FROM THE GUIDE

This is to certify that the project report titled **"Design and Development of Real-time Chat Application"** is the Bonafide work which is carried out by **Mr. Syed Awaiz** bearing **Register no: U03EH23S0082** in partial fulfilment of the requirement for the award of ***BCA degree*** of Bangalore University, under my Guidance and Supervision.

The Project report submitted by her has been successfully completed and reflects her hard work and sincere effort.

Place: Bengaluru
Date:

Dr. Sunetra Chatterjee
Project Guide

CERTIFICATE OF ORIGINALITY

This is to certify that the project report titled **"Design and Development of Real-time Chat Application''** is an original work of **Mr. Syed Awaiz** bearing **Register no: U03EH23S0082** and is submitted in the partial fulfillment of the requirement of the Bangalore University for the award of ***BACHELOR OF COMPUTER APPLICATIONS***. The work was not submitted earlier to this University/Institution for the fulfillment of the requirement of a course of study. **Mr. Syed Awaiz** is guided by **Dr. Sunetra Chatterjee**, the faculty guide per the regulations of Bangalore University.

Signature of Director
Dr Sridevi V

Place: Bengaluru
Date:

OFFER LETTER OF INTERNSHIP

RSR Technologies

Internship Confirmation Certificate

This is to certify that Syed Awaiz a student of IFIM College has successfully secured an internship position with RSR Technologies.

Internship Details:

Position/Role: Full Stack Web Application

Duration: 4th June 2025 to 4th July 2025

During Internship **Syed Awaiz** will have the opportunity to gain practical experience, apply academic knowledge, and develop industry relevant skills under the guidance of experienced professionals and customers.

We look forward for a productive and rewarding internship period.

Signature

Date: 4th June 2025

Head of RSR Technologies

CERTIFICATE FROM THE COMPANY

RSR Technologies

CERTIFICATE

OF INTERNSHIP

This certificate awarded to

Syed Awaiz

From IFIM College

In recognition of his efforts and achievements in completing the 30 days internship program in

Full Stack Web Application

Conducted from 4th June – 4th July, 2025

11th July, 2025

Ramesh SP

Program Manager

ACKNOWLEDGEMENT

I owe a deep sense of gratitude to those who have contributed to the successful completion of this endeavor and take this opportunity with much pleasure to thank all the people who have helped us through the course of our journey towards producing this Dissertation report.

At the onset, I express my gratitude to the Almighty God for his abundant grace, blessings and goodwill throughout this project.

I am grateful to my internal guide **Dr. Sunetra Chatterjee,** IFIM College for his constant support, encouragement and guidance.

I am grateful to **Mr. Ramesh SP** my external company guide, who gave their valuable time for the interaction and allowed me to carry out this project.

I would also like to thank all who helped me directly or indirectly in completing this project successfully.

Place: Bengaluru

Date:

Mr. Syed Awaiz

Register No:U03EH23S0082

DECLARATION

I hereby declare that the project report titled **"Design and Development of Real-time Chat Application"** submitted in partial fulfillment of the requirement of the degree of ***Bachelor of Computer Applications*** in Bangalore University has been prepared by me during the academic year **2025-26** under the guidance of **Dr. Sunetra Chatterji**, , IFIM College.

I further declare that this Project Report is the outcome of my own efforts and that is not submitted to any other University or Institute for the award of another degree or diploma or other certificate.

Place: Bengaluru

Date:

Mr. Syed Awaiz

Register No: U03EH23S0082

PLAGIARISM REPORT

Syed Awaiz_BCA_U03EH23S0083.pdf

Centre for Developmental Education-JAGDISH SHETH SCHOOL OF MANAGEMENT

Document Details

Submission ID

trn:oid:::3618:105485049

Submission Date

Jul 23, 2025, 10:45 AM GMT+5:30

Download Date

Jul 23, 2025, 11:19 AM GMT+5:30

File Name

Syed Awaiz_BCA_U03EH23S0083.pdf

File Size

1.3 MB

42 Pages

5,446 Words

34,328 Characters

PLAGIARISM REPORT

0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

0 AI-generated only 0%
Likely AI-generated text from a large-language model.

0 AI-generated text that was AI-paraphrased 0%
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely AI generated as AI generated and AI paraphrased or likely AI generated and AI paraphrased writing as only AI generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

TABLE OF CONTENTS

Chapter	Section	Title	Page No
1		**Introduction**	**1 - 3**
	1.1	Background	1
	1.2	Need for Real-time Chat Applications	1 - 2
	1.3	Overview of the MERN Stack	2 -3
2		**Project Objective and Scope**	**3**
	2.1	Objective	3
	2.2	Scope	3
3		**Feasibility Study**	**4-5**
	3.1	Technical feasibility	4
	3.2	Mechanical feasibility	4-5
4		**System Workflow**	**5-8**
	4.1	SDLC Overview	5-7
	4.2	Waterfall Model	7-8
5		**Implementation**	**8-10**
	5.1	Frontend using React.js	8-9
	5.2	Backend with Node.js and Express.js	9-10
	5.3	Real-time messaging with Socket.io	10
6		**Testing**	**11-14**
	6.1	Testing Strategy	11-12
	6.2	Deployment Overview	12-14

7		**Input and Output**	**14**
	7.1	Input	14
	7.2	Output	14
8		**Hardware and Software Requirements**	**14-15**
	8.1	Hardware Specifications	14-15
	8.2	Software Requirements	15
9		**Source code**	**16-22**
	9.1	Project Structure	16
	9.2	Backend Code	16-19
	9.3	Frontend Code	19-22
10		**ER Diagram**	**23**
11		**Use Case Diagram**	**24**
12		**Screenshots**	**25-27**
		Conclusion	**28-29**
		Bibliography	**30**

List of Figures

Figure no	Page no
Figure 1.0	**2**
Figure 1.1	**7**
Figure 1.2	**25**
Figure 1.3	**25**
Figure 1.4	**26**
Figure 1.5	**26**
Figure 1.6	**27**
Figure 1.7	**27**

Executive Summary

Throughout my internship, I had the chance to develop a Real-Time Chat Application based on the MERN Stack, a contemporary full-stack development technology made up of MongoDB, Express.js, React.js, and Node.js. This project provided me with significant experience with actual software development, full-stack integration, and team collaboration. It hugely improved both my technical skills and my knowledge of how web applications are designed, developed, and deployed.

The main goal of this internship was to create and develop a real-time messaging application where users can instantly talk with each other using text, incorporating features like support for emojis, user authentication, and a seamless interactive interface. The application replicates popular chat applications like WhatsApp or Messenger but on a simplified, personalized level for training and learning purposes.

During the initial part of the internship, I concentrated on learning the MERN stack architecture and on setting up the development environment. I started with the frontend development through React.js, where I gained the knowledge to build responsive user interfaces utilizing components, hooks, and conditional rendering. I developed functionalities such as user sign-up, login, chat windows, and the contact list. For backend functionality, I employed Node.js and Express.js to manage API requests, implement authentication systems, and store user sessions securely with JWT (JSON Web Tokens).

For the real-time chat interface, I included Socket.io, which facilitated real-time message exchange among users. This necessitated familiarization with WebSocket protocols and event-driven communication. I included features such as:

- Real-time messaging among multiple users.
- Message timestamps and delivery notifications.
- Online/offline user status tracking.
- Basic emoji and media support.

On the database front, I utilized MongoDB to hold user information and chat histories. I learned how to schema-design using Mongoose, handle data relationships, and execute CRUD

operations efficiently.

Learning to handle frontend and backend logic together was one of the most important aspects of the project. Balancing data flow between the server and client while maintaining real-time synchronization was a main difficulty. By persistent debugging, constant feedback from mentors, and constant collaboration with other team members, I was able to overcome these challenges and improve my problem-solving strategy.

I also studied deployment options towards the latter part of the internship and deployed the application on platforms such as Render and Vercel, learning how to configure environments, handle CORS policy, and cloud-based deployment.

This internship not only improved my programming and analytical skills but also enhanced my confidence in team work and managing actual project timelines and expectations. It educated me to value clean, maintainable code, adequate documentation, and proper communication to technical as well as non-technical stakeholders.

In summary, this internship has provided me with a strong foundation for my future career as a full-stack developer. It provided me with practical experience in using industry-standard tools and technologies, enabled me to implement a real-time working application from the ground up, and motivated me to seek further knowledge in software engineering and real-time systems. I am thankful for the guidance provided to me during this internship and eager to implement what I've learned in future work opportunities.

INTERNSHIP DAY BOOK

Name of Student: Syed Awaiz

Program: Bachelor of Computer Applications

Semester: IV

UUCMS No. U03EH23S0082

Name of Faculty Guide : Prof Sunetra Chatterjee

Name of Industry Mentor(IM) : Ramesh SP

Contact IM : 7760594868

Title of the project: Real-chat Application using MERN Stack (Full Stack Web Development)

Date	Tasks/ Work Completion
02/06/ 2025	Internship Onboarding
03/06/2025	1st meeting with Program Head
04/06/2025	Understood the project overview and requirements.
05/06/2025	Installed React JS and created the base frontend app.
06/06/2025	Installed supporting libraries like React Router, Axios, and Tailwind CSS.
07/06/2025	Designed the chat interface layout.
09/06/2025	Made the UI responsive using CSS and Tailwind.
10/06/2025	Tested and refined UI components.
11/06/2025	Finalized the basic frontend design. Checked all components and routes were working fine.
12/06/2025	Started backend setup using Node.js. Initialized npm and created the server file. Tested the server using a basic route

13/06/2025	Installed Express JS and created API routes. Set up middleware and sample endpoints.
14/06/2025	Created route structure for users and messages. Worked on understanding REST APIs.
16/06/2025	Faced issues with API routing and fixed them. Tested user and message routes locally.
17/06/2025	Created controllers for better code structure. Began separating logic from routes to controllers.
18/06/2025	Integrated dummy data responses in APIs. Worked on request and response handling.
19/06/2025	Handled error responses and status codes. Reviewed backend logic and optimized routes.
20/06/2025	Installed MongoDB and connected it locally
23/06/2025	Faced issues in MongoDB connection setup. Resolved by correcting connection string and using .env file.
24/06/2025	Created Mongoose models for users and messages. Worked on schema structure and validation. Inserted and fetched test data in MongoDB
25/06/2025	Connected API routes with MongoDB.
26/06/2025	Improved data handling with proper error messages. Pushed the complete project code to GitHub
27/06/2025	Deployed the app on Railway. Faced minor issues with build settings, which were resolved.
30/06/2025	Verified deployed app UI and APIs. Tested database connectivity on the live server. Final review and testing phase. Prepared final project summary and report.

Weekly Reports

Name of Student: Syed Awaiz

Program: Bachelor of Computer Applications

Semester: IV

UUCMS No: U03EH23S0082

Name of Faculty Guide: Prof Sunetra Chatterjee

Name of Industry Mentor (IM): Ramesh SP

Contact of IM: 7760594868

Title of the Project: Real-time Chat Application using MERN Stack

Reporting Week: Week 1 (June 2 – June 7, 2025)

Sl. No.	Details required	Description
1	Targets/ Tasks assigned at the inception of internship: (brief about each of task, expectations and timelines)	The primary focus for Week 1 was to understand the MERN stack architecture and set up the development environment. Tools like Visual Studio Code, Node.js, MongoDB, and Postman were installed.
2	Brief description of work done, tasks or targets achieved	Initialized both frontend (React) and backend (Node.js & Express) folder structures. Successfully installed all required dependencies and created a GitHub repository for version control
3	Challenges faced if any and measures taken to overcome it	Faced errors in environment variable setup and version conflicts between Node.js and MongoDB. Solved them by reinstalling dependencies and reviewing documentation.
4	Progress till date	Completed setup of the basic MERN stack project with client and server directories. Connected MongoDB database locally and tested sample routes using Postman.

Reporting Week: Week 2 (June 9 – June 14, 2025)

Sl. No.	Details required	Description
1	Targets/ Tasks assigned at the inception of internship: (brief about each of task, expectations and timelines)	The goal for Week 2 was to design and develop the frontend of the chat application using React.js. This included building user interface components such as login, registration, and chat screens
2	Brief description of work done, tasks or targets achieved	Created responsive UI components using React functional components and Hooks. Added navigation between pages using React Router. Designed basic form validations for login and signup pages
3	Challenges faced if any and measures taken to overcome it	Faced issues while managing states for forms and error messages. Solved them by using use State and use Effect effectively. Took guidance from mentors and online documentation.
4	Progress till date	Completed frontend design of login, register, and chat pages. UI is fully responsive and visually clean. Frontend is ready to be connected with backend APIs for full functionality.

Reporting Week: Week 3 (June 16 – June 21, 2025)

Sl. No.	Details required	Description
1	Targets/ Tasks assigned at the inception of internship: (brief about each of task, expectations and timelines)	The main task for Week 3 was to develop the backend using Node.js, Express, and MongoDB. This included building APIs for user authentication and message handling.
2	Brief description of work	Created Express server and defined routes for user

	done, tasks or targets achieved	registration, login, and chat. Used Mongoose to design schemas for storing user and message data. Connected the backend to MongoDB and tested all routes using Postman.
3	Challenges faced if any and measures taken to overcome it	Encountered issues with data not being stored properly in MongoDB. Fixed it by updating schema validation and testing different data formats in Postman. Also learned to use async/await efficiently to avoid callback issues.
4	Progress till date	Backend APIs are successfully developed and tested. MongoDB is storing user and message data correctly. Frontend and backend are partially integrated and communication is working on local setup.

Reporting Week: Week 4 (June 22– June 28, 2025)

Sl. No.	Details required	Description
1	Targets/ Tasks assigned at the inception of internship: (brief about each of task, expectations and timelines)	The focus for Week 4 was to implement real-time chat functionality using Socket.io, complete frontend-backend integration, and perform final testing using Postman.
2	Brief description of work done, tasks or targets achieved	Integrated Socket.io into the backend and established a connection with the front end. Enabled real-time message sending and receiving between users. Tested all APIs and chat features in Postman and browser environment.
3	Challenges faced if any and measures taken to overcome it	Faced Socket disconnection issues during browser refresh. Solved it by implementing reconnection logic and managing server-client events carefully. Also encountered CORS issues, which were fixed

		using Express middleware
4	Progress till date	Full real-time chat functionality is now active. Users can register, log in, and exchange messages instantly. APIs are tested and working via Postman. The project is stable and ready for final review or demonstration.

1. Introduction

1.1 Background

With the arrival of technology in the contemporary world, communication is fast, convenient, and interactive. One of the most commonly used media of communication is real-time chat applications. With chat applications, the users can send the messages in real-time, and it makes conversation simple and effective. Chat applications have entered into daily life, whether it is personal or business communication.

Real-time chat app history has come a long way with the advent of new web technologies. MERN Stack is one of the robust technology stacks used, which represents MongoDB, Express.js, React.js, and Node.js. The full-stack JavaScript tech is used for developing dynamic and quick web applications. It allows developers to employ a single programming language (JavaScript) for both front-end and back-end, which enhances productivity as well as consistency in code.

The objective of this project is to develop a real-time chat application using the MERN Stack with user authentication, real-time messaging using Socket.io, chat history, profile photos, and end-to-end secure conversation. The objective is to develop a simple yet effective application that exhibits the capabilities and flexibility of the MERN stack in building full-stack web applications with real-time functionalities.

1.2 Need for Real-Time Chat Applications

In today's era of immediate messaging systems based on page reloads or gradual updates do not satisfy the needs of users who require a smooth and instant experience. Live chat applications satisfies this need by incorporating live chat between members with no lag.

They are being widely utilized in different industries, including : Customer support, Online education Healthcare websites, Social networking. They help organization’s increase user interaction, enhance coordination, and offer improved customer support.

Apart from that, remote teams require real-time communication, so that they can effectively collaborate from different locations. The ability to send and receive messages in real-time, observe online statuses, and receive real-time notifications has become a norm in all current

day web and mobile applications. Thus, the necessity of creating real-time chat application is growing at a fast pace and hence it is a critical skill for developers and essential features for businesses.

1.3 Overview of MERN Stack

The MERN Stack is a strong and popular combination of technologies used to develop full-stack web applications with JavaScript used for both server-side and client-side development. MERN is an abbreviation that refers to:

a. M – MongoDB (Database)
b. E – Express.js (Backend Framework
c. R – React.js (Frontend Library)
d. N – Node.js (Server Environment)

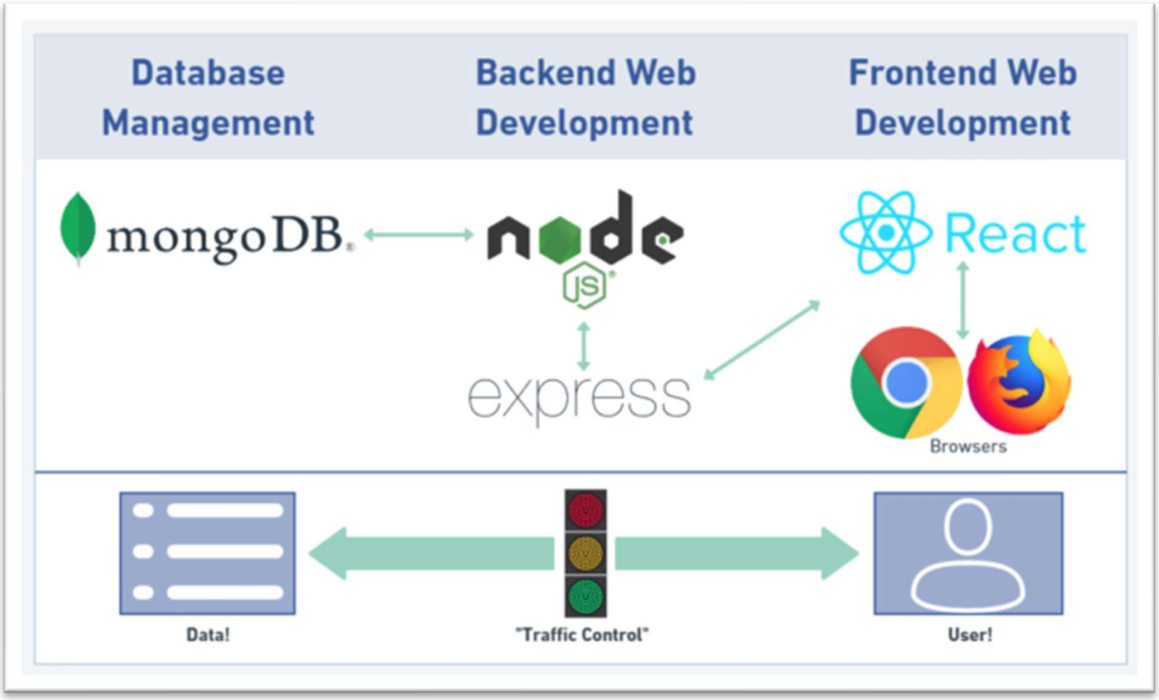


Figure 1.0

a. MongoDB:

A NoSQL document-based database that stores data in JSON-like format. It is scalable, flexible, and can be used to store chat messages, user information, and other information required for real-time applications.

b. Express.js:

A lightweight web development framework for Node.js. It eases the process of developing server-side logic and offers a fast, interactive frontend experience.

c. Node.js:

A server-side runtime environment that runs JavaScript code. It is event-driven and non-blocking, making it ideal for real-time applications. that require quick data processing and response times.

The MERN stack allows developers to build robust, scalable and real-time web applications using a single programming language—JavaScript. This improves development speed, consistency, and maintainability.

2. Project Objective and Scope

2.1 Objective

The main objective of this project is to develop and establish a real-time chat application using the MERN Stack, which includes MongoDB, Express.JS, React.JS and Node.JS.

- To incorporate user registration and authentication so that only authenticated users can be granted access to the chat
- To incorporate real-time messaging using Socket.io so that the message is displayed immediately without page reloading.
- To design a desirable and friendly chat interface to efficiently use MongoDB.
- To install and deploy a complete full-stack web application that works seamlessly on web browsers.

2.2 Scope

This project seeks to build a straightforward real-time chat program with the MERN Stack where users can send and receive messages instantly in a single conversation.

- Chat History: Saving and displaying the history of previous messages for a single conversation with MongoDB.
- Real-Time Messaging: Enabling users to send messages in real-time without page refresh, using Socket.io
- Frontend Interface: Building a beautiful and responsive chat interface using React.JS.
- Backend API: Creating RESTful APIs using Express.js and Node.js to handle user and message information.
- Profile Picture Setup: Enabling users to select and upload an avatar as a profile picture
- Deployment: Deploying applications online with tools like Heroku or Render.

3. Feasibility Study

Feasibility study is a significant component of the software development life cycle. It helps understand if a project is feasible, applicable, and viable to be done with what is accessible.
In this project, a feasibility study is conducted to try out if it is technically and market-wise feasible to build a live chat application using the MERN Stack.

3.1 Technical feasibility

MERN Stack is strong and modern full-stack technology that is most suitable for real-time web application development.
This project is technologically feasible on the grounds that:

- Developer Skillset: Technologies used (Express.JS,React.JS, MongoDB, Node.JS) are open-source technologies with strong documentation and hence easy to adopt and use by developers.
- Real-Time Communication: This enables real-time messaging through Socket.io that is lightweight and technologically viable.
- One language (JavaScript): Since JavaScript is used between the front-end and back end, the development process is made easy and efficient.
- Open-Source Tools: Everything needed is open-source on all the platforms. Lower development cost and dependency on paid service.
- Scalability: Later, the app can be scaled out to accommodate group chat or other features if required.

3.2 Mechanical feasibility

Real-time chat applications are most in demand on both a professional and individual level.
The project is market feasible because:

- Augmented Demand: Message apps are used extensively in social media, education, customer care, and e-commerce.
- Clean UI with Core Features: A simple and intuitive chat application is appealing to small firms and start-ups.
- Learning Goal: It is worthwhile for students and interns who want to learn hands-on in full-stack development.
- High Usage Pattern: An increasing number of individuals use instant, real-time communication, creating high demand for easy chat solutions.

Thus, the project is market viable particularly as a portfolio project or as a foundation for future products.

4. System Workflow

4.1 SDLC Overview

Software Development Life Cycle or SDLC is a systematic process utilized by computer programmers in order to develop high quality, effective, and cost-efficient software within a given timeframe. It is a step-by-step mechanism for planning, making, testing software, deploying software, and software maintenance

Phases of SDLC in Detail

1. Requirement Gathering and analysis This is the first and most important phase of SDLC. Here, the developer selects and procures all the required requirements from stakeholders or the client for the software. In our project, we have chosen to develop a chat application that should:

    - Allow users to register and login.
    - Allow one to one real time messaging
    - Save and restore chat history.
    - Be responsive and user friendly.

2. System Design

    System Design Based on the requirements received, the system has been developed. This comprises high-level design(architecture) and low-level design (database schemas, APIs, UI layout). We designed the following:

    - A simple architecture based on the MERN Stack.
    - A clean and minimalistic chat interface using React.
    - API controllers and routes using Express.
    - MongoDB collections to store user and message data.
    - Integration with Socket.io for real-time communication.

    This design helped understand how different modules will communicate while implementing.

3. Implementation (Coding)

   Here the application development was done. It was segregated into three broad categories of growth:

   - Frontend: Implemented using React.js with chat UI elements, login form, registration form, and message display.
   - Backend: Implemented using Node.js and Express.js for API serving, user authentication, and handling messages.
   - Database: Utilized MongoDB to store and retrieve user and chat information
   - Real-Time Layer: Used Socket.io to offer real-time messaging without page refresh.

4. Testing

   Testing is performed to detect and correct the errors identified in the application. Different types of tests were conducted:

   - Unit Testing: Modules and functions were tested.
   - Integration Testing: Front End, Back End, and Database were tested collectively to authenticate that they operate correctly
   - User Testing: Users were allowed to test the app to verify usability, looks, and actual-time functioning. The testing proved that the application is performing as expected with almost no bugs or crashes.

5. Deployment

   After successful testing, the application was deployed on a web hosting site like Heroku or Render. Deployment places the application online for access by the users. Deployment time saw minor adjustments such as UI changes or error handling optimization.

6. Maintenance in the future can include:

   With added functionality of group chat or file transfer. Correcting bugs based on user complaints. Refactoring the backend or UI for better performance. SDLC keeps the software from deviating from user expectations and works satisfactorily in real-world environments.

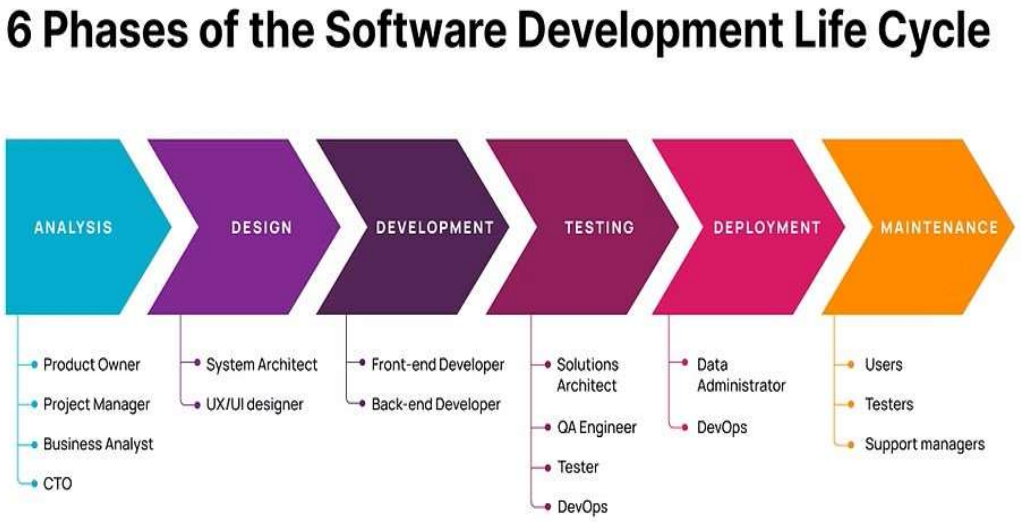


Figure 1.1

4.2 Waterfall Model

Waterfall Model is a sequential iterative software development process where each phase has to be finished before starting the next phase. For developing a Real-Time Chat Application using the MERN Stack, these phases are:

- Requirement Analysis At this stage, we enumerated all the requirements of the chat application. The above mentioned were implemented like user registration/login, 1-to-1 chat, group chat, message in real-time, and history of messages.

  Questionnaires and focus groups were utilized as the tools to understand what the app should be capable of doing.

- System Design

  The technical design and system architecture were designed.

  - Frontend: React.js for UI/UX.
  - Backend: Node.js with Express.js to manage APIs and server logic.
  - Database: MongoDB to store the user profiles, chat logs, and group information
  - Real-time communication: Used Socket.IO for real-time messaging.

  Online Database schema, REST APIs, and socket event structure were all designed in this phase.

- Implementation (Coding)
  - Development of the chat application was actually completed.
  - Frontend: Designed responsive chat UI using React.js and CSS.
  - Backend: Implemented RESTful APIs for user login and message CRUD operations using Express.js and Node.js.
  - WebSocket's: Added Socket.IO for real-time data transfer.
  - Code was module-based and version-controlled with Git and GitHub.
- Testing
  - Unit testing and integration testing were performed.
  - Verified that messages were sent/received in real-time without delay properly.
  - Verified user login, database interaction, and error handling.
  - Test report bugs were resolved.
- Deployment.
  - The application was hosted on a cloud server (e.g., Render, Vercel, or Heroku).
  - MongoDB is hosted on MongoDB Atlas.
  - DNS settings and environment variables were set securely.
  - Deployed version was reverified for scalability and performance.
- Maintenance

We tracked the app for crashes, bugs, or performance issues post-deployment. Based on consumer reviews, some improvements such as dark mode and file sharing features were mentioned for future releases. A regular maintenance and database backup were scheduled.

5.Implementation

The implementation phase is where the application is developed based on the system design. The application was created using the MERN Stack, which consists of MongoDB, Express.js, React.js, and Node.js. This project also employs Socket.io in an attempt to offer real-time messaging.

The application was created on three major components:

1. Frontend using React.js
2. Backend using Node.js and Express.js
3. Real-time communication using Socket.io

5.1 Frontend using React.js

The frontend is that part of the app which the users will be utilizing. It was built using

React.js, a robust JavaScript library used to create dynamic and adaptable user interfaces. React uses a component-based model, which helps to split the UI into small, iterable pieces of code-referred-to-as components.

Major Features Implemented in the Frontend:

- Login and Registration Forms: Registration is possible by the user by entering his name, email, and password. Login form logs users in and directs them to the chat interface. Input validation was employed to prevent blank fields or invalid format.
- Chat Interface: Plain and simple chatting window in which users can compose and send messages. Messages are displayed in real time as soon as they are received. Interface shows timestamps, sender name, and message body.
- Message Component: Message shown in single styled box. User messages are displayed on the right; others' messages are displayed on the left.
- User Interface Design: CSS and frameworks like Tailwind or Bootstrap were used for responsiveness, spacing, layout, and fonts. Media queries render the chat responsive on desktop as well as mobile screens.
- API Calls: Axios or Fetch API was used to make a call to the backend for login, registration, and fetching chat history. React state and props were used to handle user input and display data dynamically.

5.2 Backend with Node.js & Express

Backend will handle business logic, data processing, and data return to frontend. It is developed using Node.js, a JavaScript runtime environment, and Express.js, a lightweight and fast web framework.

Key Backend Features:

- User Authentication: Handled using JWT (JSON Web Tokens). Passwords are stored encrypted with crypt for security reasons. Private routes are secured using middleware.
- RESTful API Endpoints: POST /register – for creating new users. POST /login – for logging in existing users. GET /messages – for fetching older messages. POST /messages – for saving a new message in the database.
- Database Operations: Used MongoDB as a database. Used Mongoose to define schemas for User and Message collections. Data is being saved in JSON-like documents.

- Middleware Implementation: Applied custom middleware to verify the JWT token. Restricts chat APIs access to logged-in users only.
- CORS Configuration Configured CORS (Cross-Origin Resource Sharing) to allow frontend and backend to communicate through messages on other ports.

5.3 Real-Time Messaging with Socket.io

Real-time messaging is the basic functionality of this project. It was achieved using Socket.io, which is a JavaScript library for real-time web applications. Socket.io provides bi-directional communication between client and server via Web Sockets.

How Real-Time Messaging Works:

- When a user signs in, the frontend establishes a connection with the server via Socket.io.
- When the message is typed and sent, it is sent to the server using socket. emit ().
- The server receives the message and sends it to the receiver using socket. broadcast. emit ().
- The receiver receives the message instantly and it is shown in the chat window.
- The message is also stored on the MongoDB database.

Socket Events Used

- socket. On('connect'): Triggered when a client connects to the server.
- socket. On ('send Message', data): Receives the message from the sender.
- socket. Emit ('receive Message', data): Sends the message to the targeted user.
- socket. Disconnect (): Handles when a user leaves or disconnects from the application.
- Socket.io does not have any message delays and offers an untainted chatting experience, just like Messenger or WhatsApp.

Conclusion of Implementation

The implementation had careful integration of all layers—frontend, backend, and real-time messaging. Each module was coded and sewn together using APIs and socket events to develop a fully functional, real-time chat application with the most up-to-date features and responsive design.

6.Testing

Two very important final stages are testing and deployment in software development life cycle. Testing makes sure that the application is functioning in the expected manner, while deployment releases the application to end users. This sentence outlines the test methods utilized and how the app was deployed to a cloud environment.

6.1 Testing Strategy

Testing refers to the process of finding a system in order to pinpoint any bugs, defects, or lost functionality prior to deployment with users. An effective test plan was in place so that the real-time chat system would be operating suitably, securely, and effectively.

Types of testing done

1. Unit Testing: Unit testing refers to testing individual units or functions of the application in silos.

   For this project:

   - All React components (e.g., input fields, message boxes, forms) were tested to verify that they are rendering properly.
   - Backend controller actions were input/output tested with test data.
   - For instance, to verify if the login process throws an exception for invalid credentials.

2. Integration Testing: Integration testing ensures that different parts of the system function properly when integrated together. In this project, the frontend was integrated with the backend APIs via Axios to transmit and receive data.

   We tried:

   - Sending a message out and checking whether it shows on the recipient's screen.
   - Checking whether a newly created user can sign in and navigate to the chat screen.
   - Checking whether the chat messages are stored in the MongoDB database.

3. Real-Time Functionality Test: Since it is an internet chat software, additional testing was done for live messaging transmission. This included:

   - Two logins by two different devices or browsers.
   - Sending a message from one user to another and verifying if it appears immediately on the other user's display.
   - Testing Socket.io connection, disconnection, and reconnection events.

- Messages were found to be sent without page reload and with very little delay.

4. User Interface (UI) Testing: UI testing is conducted to ensure that the user interface is clean, easy to navigate, and responsive across various screen sizes.

   We checked:

   - Login and registration form validations.
   - Message alignment, colour, and spacing.
   - Mobile responsiveness through developer tools.
   - Error messages for invalid input or failed API call.

5. Security Testing: Security testing is a must for any application.

   We ensured:

   - Passwords are stored encrypted through crypt.
   - JWT tokens are used to secure private routes.
   - Unauthorised users are not given access to the chat screen.
   - Test requests to fetch data without a token were rejected by middleware.

6. Bug Fixing and Feedback Testing: After in-house tests the application was tried by multiple users to gather feedback.

   Some problems found and resolved:

   - Duplicate posted messages due to socket misconfiguration.
   - Login form not clearing after logout.
   - Alignment issues on small screens on mobile phones.

6.2 Deployment Overview

After testing, the final working version of the project was deployed to put it live online. The objective of deployment is to get the frontend and backend on secure cloud servers to enable users to access the app through a web browser. Deployment Platforms Utilized :

- Backend Deployment

  Platform: Render /Heroku

  - Express.js server and MongoDB connection were employed as a Node.js service.
  - Environmental variables (like database URI and JWT secrets) were added securely through the platform's environment settings.

- Frontend Deployment

  Platform: Vercel / Netlify

  - React.js build directory was uploaded and pointed to a GitHub repository.
  - Automatic deployment was triggered once final code was committed to GitHub.

  Steps Involved in Deployment :

  - Backend Deployment
    - Create a Node.js backend for the application.
    - Set MongoDB URI from Atlas.
    - Included. env variables (PORT, DB_URI, JWT_SECRET).
    - Deployed on Render/Heroku using npm start command.
  - Frontend Deployment

    Made a production build with:

    - Arduino Duplicate Adjust npm run build
    - Bound GitHub repository to Netlify / Vercel.
    - Configured build command and publication directory.
    - Served with a custom domain if required. ¬ Linking Frontend and Backend
    - Updated frontend Axios base URL to refer to the deployed backend server.
    - Allowed CORS configurations on the server-side to accept frontend domain requests.
  - Post-Deployment Testing

    Full application testing was re-run after deployment to deliver the following:

    - Application loads flawlessly in all browsers.
    - Login, chat, and messages are working normally. •\tSocket.io still works after deployment.
    - Page reloads won't make the authentication null. ¬\tAdvantages of Deployment
    - The application can be accessed anywhere and anytime.
    - No testing and deployment are two very important final steps in the software development cycle.

- Testing ensures that the application works as required, while deployment sends the application to the users.

7. Input and Output

This section outlines the main inputs, outputs, and main functionality of the Real-Time Chat Application developed with the MERN Stack (MongoDB, Express.js, React.js, Node.js) and Socket.io for real-time.

7.1 INPUT

- They may log in using their password and email.
- They can log in using the same email and password through which they have signed up.
- The chat window can be used freely by the users and they can interact with anyone who is registered on the app.
- The users can log off the chat window at any time.

7.2 OUTPUT

- Other active users are made visible to the users.
- They can get information and read messages in real-time.
- They can also communicate via emojis and experience real-time messaging without any interruptions.

8. Hardware and Software Requirements

In order to create and execute the Real-Time Chat Application on the MERN stack (MongoDB, Express.js, React.js, Node.js), the following hardware and software specifications are suggested:

8.1 HARDWARE SPECIFICATIONS

1. Processor:
   1.6 GHz or higher processor computer (Dual-core or higher for quick development and testing).
2. Memory (RAM): Minimum:

1 GB (32-bit) or 2 GB (64-bit) memory.

3. Recommended:

   4 GB or more, particularly when code editors and local servers are operating simultaneously.

4. Hard Disk:

   At least 20 GB of free disk space.

5. Recommended:

   SSD for enhanced file access and build speed.

6. Hard Drive Speed:

   5400 and above (SSD is best for optimal performance).

8.2 SOFTWARE REQUIREMENTS

1. Operating System:
   - Windows Operating System (Windows 7 / 10 / 11) and above.
   - Linux and macOS compatible on requirement basis.
2. Code Editor:
   - Visual Studio Code (VS Code) – for coding and developing frontend and backend.
   - Add-ons such as Prettier, ESLint, and MongoDB tools for improved development.
3. Database Server:
   - MongoDB Server – NoSQL database to hold user information and chat logs.
   - MongoDB Compass (optional) for graphical interface interaction with the database.
4. API Testing Tool
   - Postman – to test REST APIs utilized in backend development.
5. Runtime Environment:
   - Node.js – to build the Express server and handle backend operations.
   - Comes with npm (Node Package Manager) to install dependencies needed.
6. Other Tools (Optional but Helpful):
   - Git & GitHub – for group development & version control.
   - Browser (Chrome/Firefox) – to run the frontend React app. Socket.io – for real-time user connectivity.

9. Source Code

9.1 Project Structure:

```
mern-chat-app/
├── backend/
│   ├── config/
│   ├── controllers/
│   ├── middleware/
│   ├── models/
│   ├── routes/
│   ├── .env
│   ├── server.js
│   └── package. Json
├── frontend/
│   ├── public/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── App.js
│   │   ├── index.js
│   └── package. Json
```

9.2 Backend Code :

- backend/server.js

```
const express = require('express');
const http = require('http');
const mongoose = require('mongoose');
const dotenv = require('dotenv');
const cors = require('cors');
const socketIo = require('socket.io');
const authRoutes = require('./routes/authRoutes');
const messageRoutes = require('./routes/messageRoutes');
const { verifyToken } = require('./middleware/auth');
dotenv.config();
```

```
const app = express();
const server = http.createServer(app);
const io = socketIo(server, {
  cors: { origin: '*' }
});
app.use(cors());
app.use(express.json());
mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('MongoDB connected'))
  .catch(err => console.log(err));
// Routes
app.use('/api/auth', authRoutes);
app.use('/api/messages', verifyToken, messageRoutes);
// Socket.io Events
io.on('connection', (socket) => {
  console.log('New client connected');
  socket.on('sendMessage', (data) => {
    io.emit('receiveMessage', data);
  });
  socket.on('disconnect', () => {
    console.log('Client disconnected');
  });
});
const PORT = process.env.PORT || 5000;
server.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

- backend/models/User.js :

```
const mongoose = require('mongoose');
const user Schema = new mongoose. Schema({
  username: String,
  password: String,
});
```

```
module.exports = mongoose.model('User', userSchema);
```

- backend/models/Message.js :

```
const mongoose = require('mongoose');
const messageSchema = new mongoose.Schema({
  sender: String,
  text: String,
  timestamp: { type: Date, default: Date.now },
});
module.exports = mongoose.model('Message', messageSchema);
```

- backend/routes/authRoutes.js

```
const express = require('express');
const router = express.Router();
const User = require('../models/User');
const jwt = require('jsonwebtoken');
router.post('/register', async (req, res) => {
  const { username, password } = req.body;
  const newUser = new User({ username, password });
  await newUser.save();
  res.send({ message: 'User registered' });
});

router.post('/login', async (req, res) => {
  const { username, password } = req.body;
  const user = await User.findOne({ username, password });
  if (!user) return res.status(400).send({ error: 'Invalid credentials' });
  const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET);
  res.send({ token, username: user.username });
});
module.exports = router;
```

- backend/middleware/auth.js :

```
const jwt = require('jsonwebtoken');
```

```
function verifyToken(req, res, next) {
  const token = req.headers['authorization'];
  if (!token) return res.status(403).send({ error: 'No token provided' });
  jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
    if (err) return res.status(500).send({ error: 'Failed to authenticate token' });
    req.userId = decoded.id;
    next();
  });
}
module.exports = { verifyToken };
```

- .env

```
MONGO_URI=mongodb://localhost:27017/
mernchat
JWT_SECRET=your_jwt_secret
```

9.3 Frontend Code :

- frontend/src/index.js :

```
import React from 'react';
import ReactDOM from 'react-dom';
import App from './App';
ReactDOM.render(<App />, document.getElementById('root'));
```

- frontend/src/App.js :

```
import React, { useState } from 'react';
import Login from './pages/Login';
import Chat from './pages/Chat';
function App() {
  const [user, setUser] = useState(null);
  return user ? (
    <Chat user={user} />
  ) : (
```

```
    <Login setUser={setUser} />
  );
}
export default App;
```

- frontend/src/pages/Login.js :

```
import React, { useState } from 'react';
import axios from 'axios';
function Login({ setUser }) {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const login = async () => {
    const res = await axios.post('http://localhost:5000/api/auth/login', { username, password });
    localStorage.setItem('token', res.data.token);
    setUser({ username });
  };
  return (
    <div>
      <h2>Login</h2>
      <input placeholder="Username" value={username} onChange={(e) => setUsername(e.target.value)} /><br />
      <input placeholder="Password" type="password" value={password} onChange={(e) => setPassword(e.target.value)} /><br />
      <button onClick={login}>Login</button>
    </div>
  );
}
export default Login;
```

- frontend/src/pages/Chat.js :

```
import React, { useEffect, useState } from 'react';
import io from 'socket.io-client';
```

```
import axios from 'axios';
const socket = io('http://localhost:5000');
function Chat({ user }) {
 const [messages, setMessages] = useState([]);
 const [text, setText] = useState('');
 useEffect(() => {
  const fetchMessages = async () => {
   const res = await axios.get('http://localhost:5000/api/messages', {
    headers: { Authorization: localStorage.getItem('token') }
   });
   setMessages(res.data);
  };
  fetchMessages();
  socket.on('receiveMessage', (message) => {
   setMessages((prev) => [...prev, message]);
  });
  return () => socket.off('receiveMessage');
 }, []);
 const sendMessage = async () => {
  const message = { sender: user.username, text };
  await axios.post('http://localhost:5000/api/messages', message, {
   headers: { Authorization: localStorage.getItem('token') }
  });
  socket.emit('sendMessage', message);
  setText('');
 };
 return (
  <div>
   <h2>Chat</h2>
   <div style={{ maxHeight: '300px', overflowY: 'scroll' }}>
    {messages.map((msg, i) => (
     <p key={i}><b>{msg.sender}:</b> {msg.text}</p>
    ))}
   </div>
```

```
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <button onClick={sendMessage}>Send</button>
    </div>
  );
}
export default Chat;
```

To Run the Project

1. Backend:

   cd backend

   npm install

   node server.js

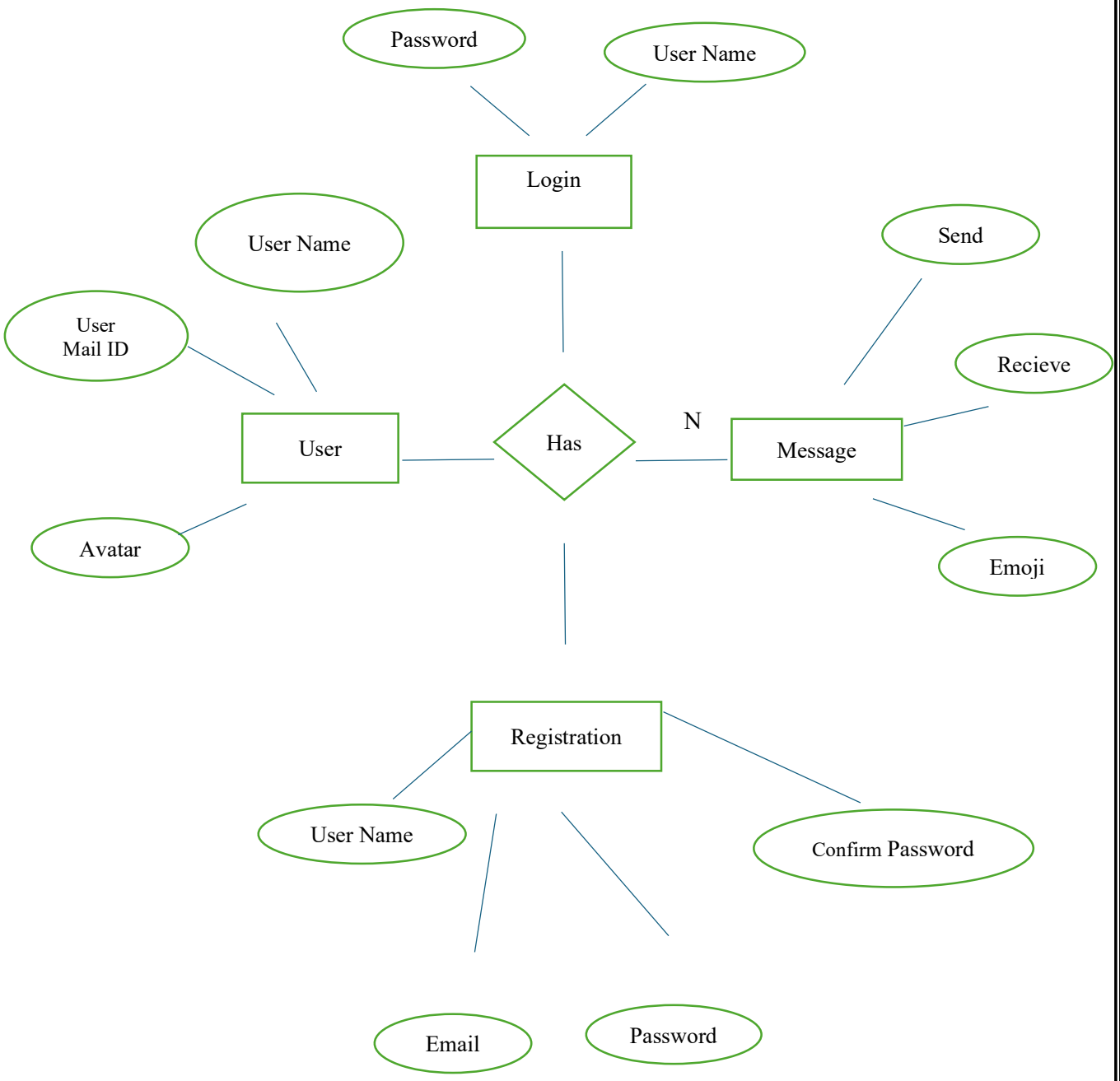
2. Frontend:

   cd frontend

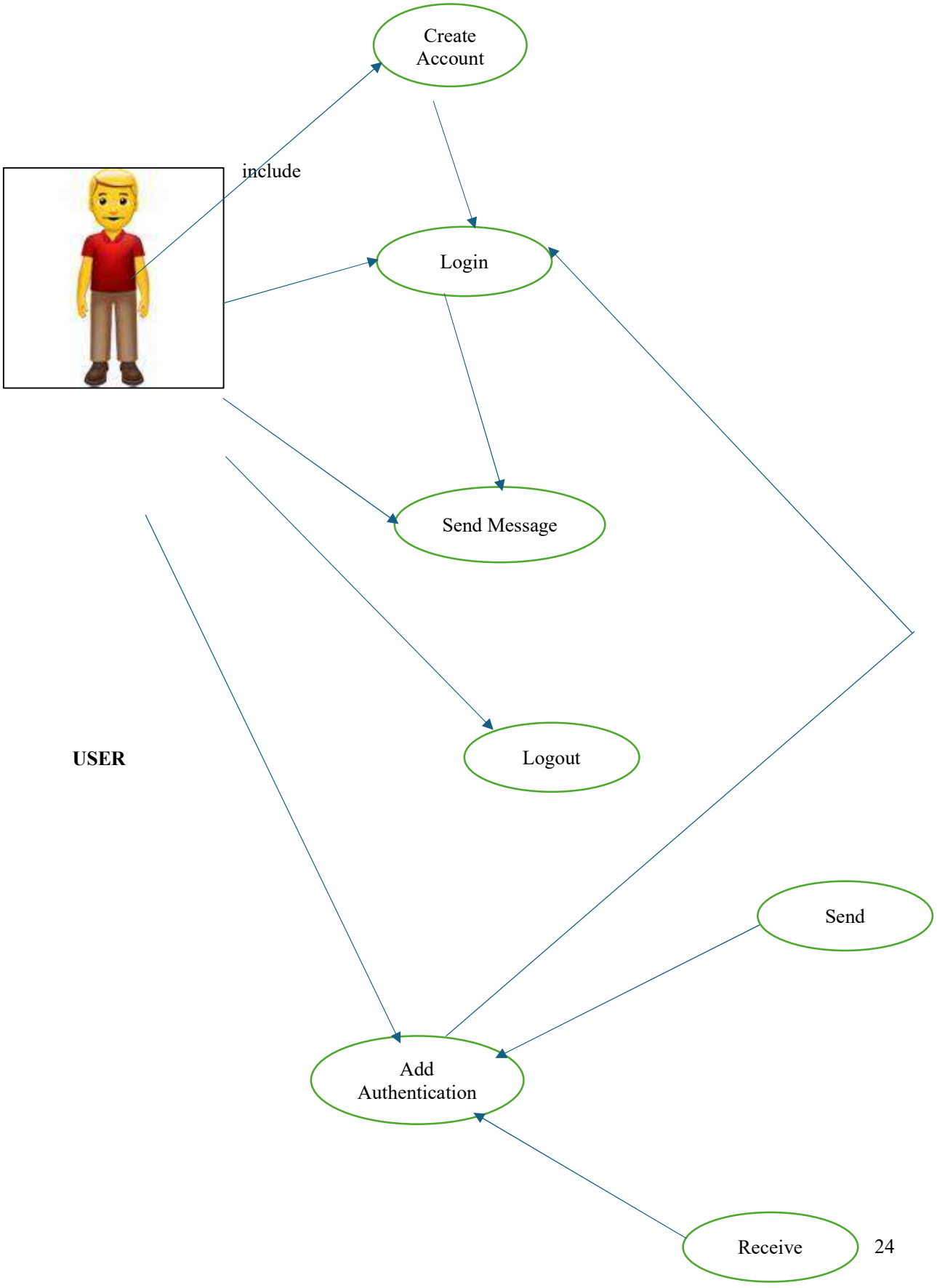
   npm install

   npm start

10. ER Diagram



11. Use Case Diagram



12. Screenshots

12.1 Group+ Notification PNG

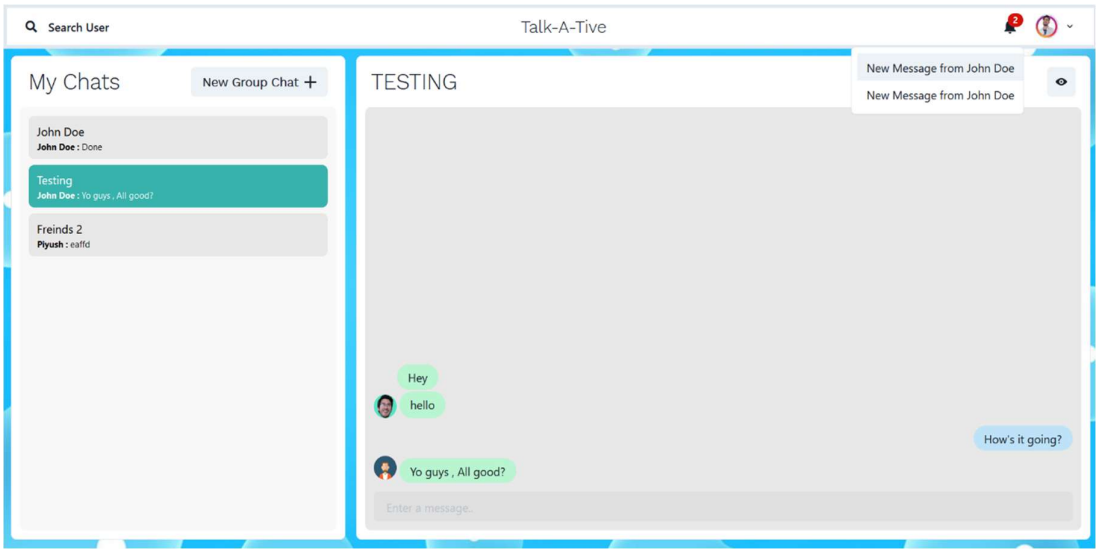


Figure 1.2

12.2 Login Page PNG

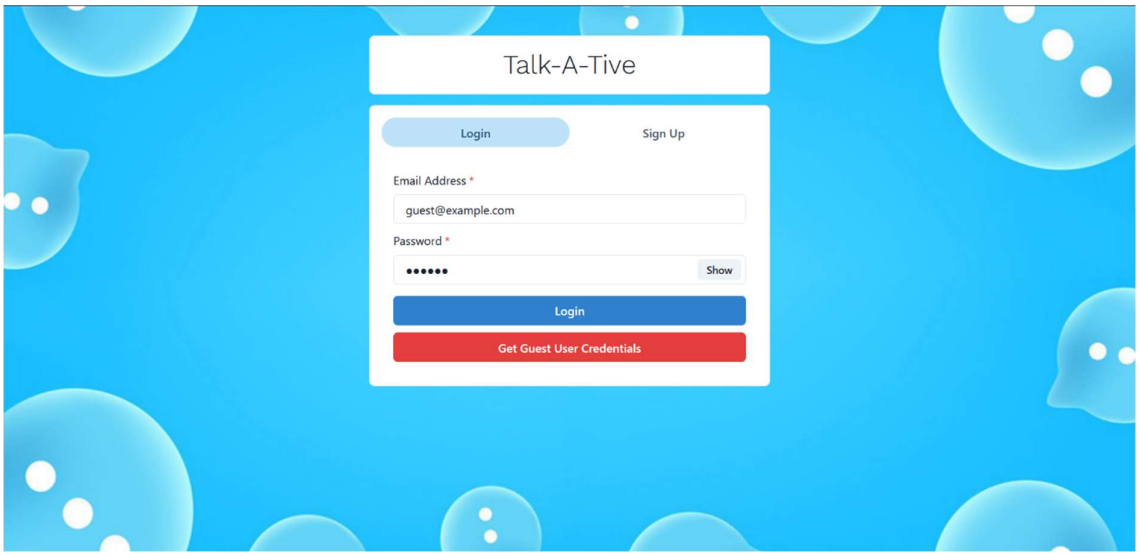


Figure 1.3

12.3 Signup PNG

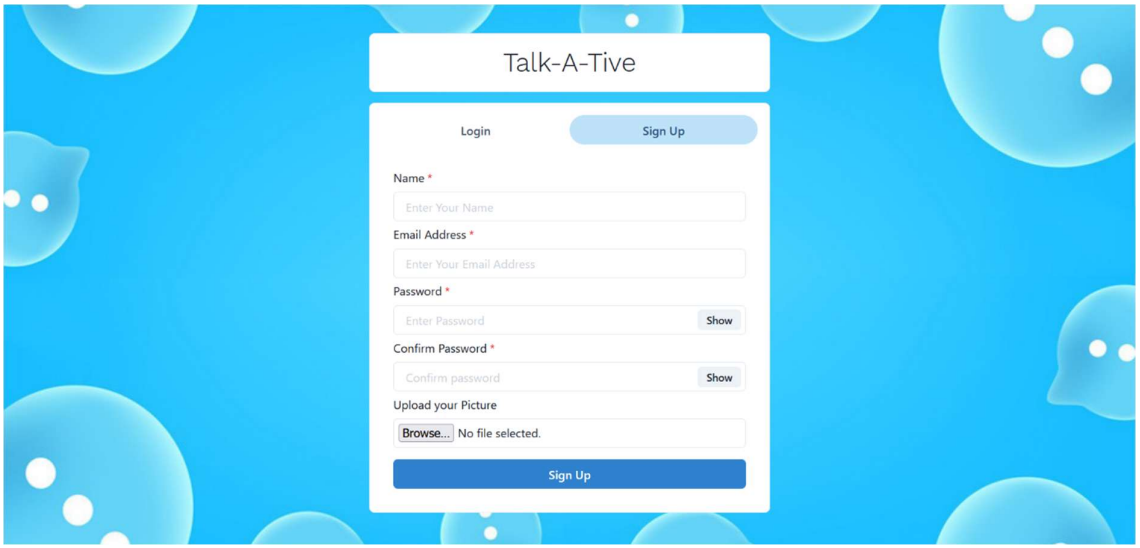


Figure 1.4

12.4 Search PNG

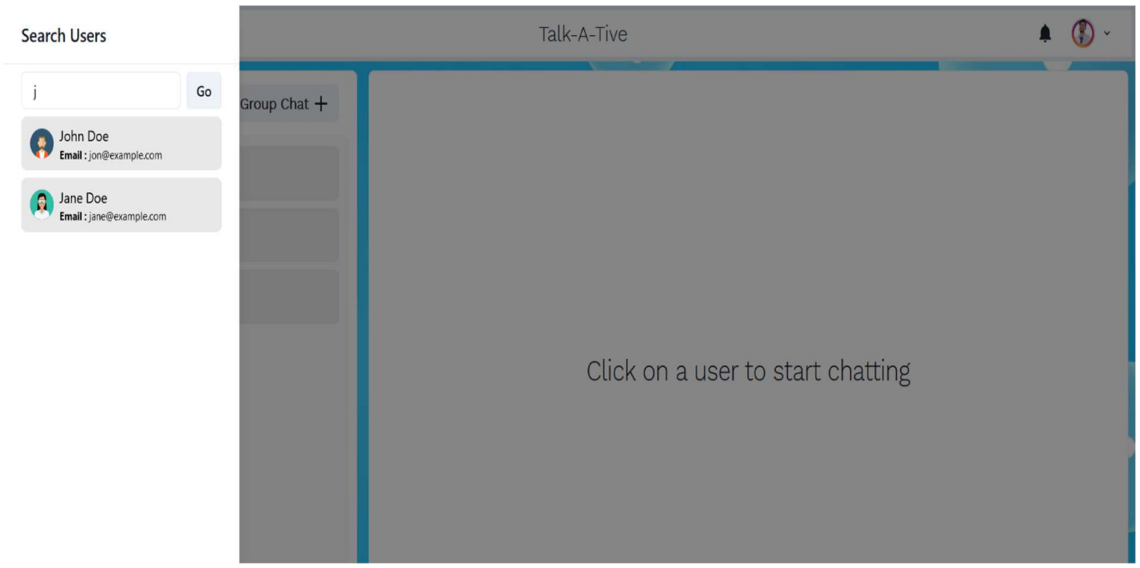


Figure 1.5

12.5 Real-time PNG

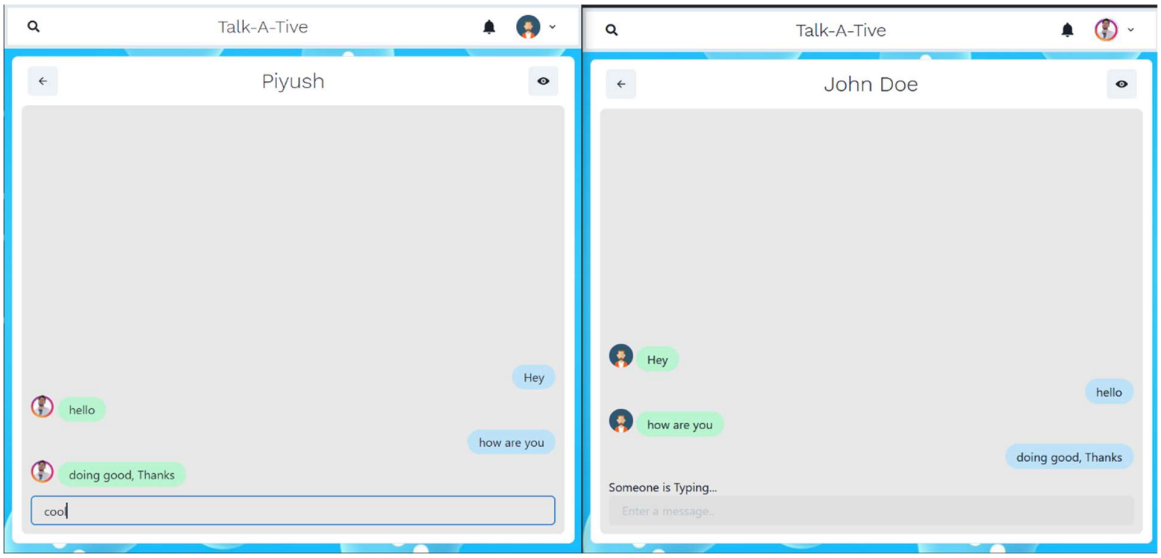


Figure 1.6

12.6 Main screen PNG

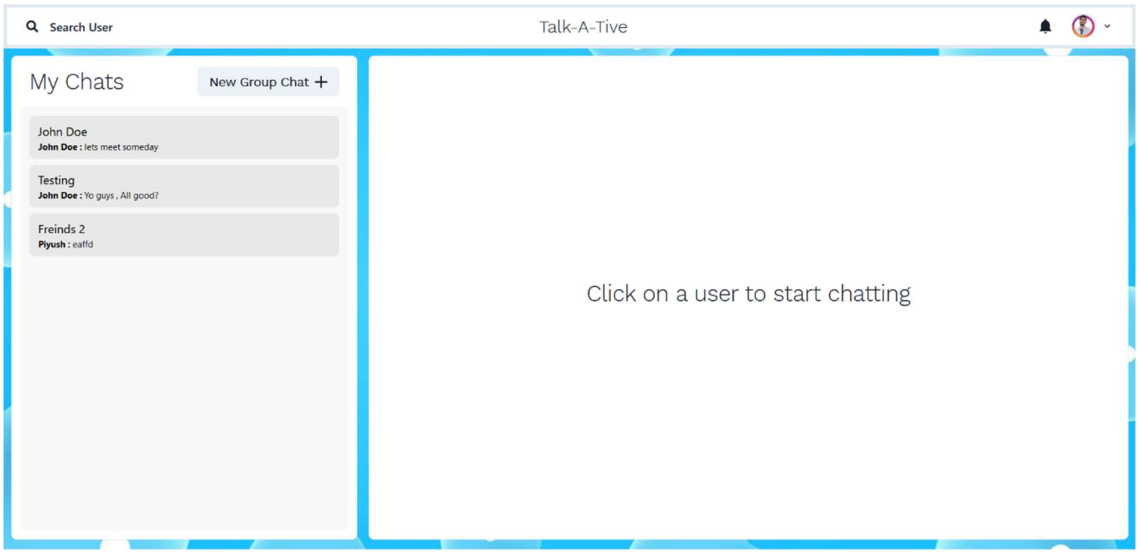


Figure 1.7

Conclusion

Development of the Real-Time Chat Application using the MERN Stack has proved to be an incredibly enlightening and worthwhile experience. The project provided a real opportunity to apply some of the most widely used web technologies of the modern age—MongoDB, Express.js, React.js, and Node.js—along with Socket.io for enabling real-time communication. The project was not only a learning platform but also an imitation of how real-world software development is carried out, especially in full-stack web development and client-server design. Throughout the development process, I learned to build a project from the ground up—starting from requirement analysis and system design to frontend and backend coding, API testing, and integration of real-time features. Each phase of the project contributed significantly to the development of my technical and problem-solving skills. Proper implementation of user authentication, state management, socket connections, and message broadcasting indicates a proper understanding of full-stack development concepts.

In bringing together frontend and backend aspects was the greatest project lesson, as seamless data exchange between server logic and client-facing interface is essential. Using React.js for the frontend enabled me to build a responsive user interface, and Node.js and Express.js in the backend gave me a solid base to manage routes, sessions, and data manipulation. MongoDB, a NoSQL database, was essential in efficiently storing and retrieving chat messages and user information.

Another of the most exciting aspects of the project was incorporating Socket.io. The core of any chat app is live messaging, and learning how to handle socket connections, emit events, and handle live communications was an eye-opening experience. This gave me a first-hand understanding of how new apps like WhatsApp, Slack, and Messenger function behind the scenes.

I also faced a couple of real-world issues during the internship. They were CORS issues, working with asynchronous data streams, API error handling, and debugging socket disconnects. Through endless debugging, documentation, and using common sense, I was able to debug those issues and improve the stability and performance of the application. Testing and verifying all backend APIs through Postman also helped me understand server-side development and request handling better.

Apart from technical training, this project also taught me the importance of writing clean, readable, and modular code. I was capable of learning to embrace best practices such as dividing front-end and back-end logic, using environment variables for security reasons, and

being consistent when it comes to design and development. Building this application also improved my ability to document what I do and cooperate with others when required.

This internship project has given a strong foundation for my career in web development and encouraged me to work on full-stack applications confidently. What I have learned—such as designing RESTful APIs, database operations, handling real-time data, and deployment and testing with Postman—is certain to be of great help in my future academic and professional endeavors.

In summary, the Real-Time Chat Application project was not just an academic assignment but a learning exercise from end to end that integrated various parts of full-stack development. It has served to bridge the gap between knowledge and practice for me. I am highly grateful to have the privilege of undertaking this project, and I hope to apply these skills in even more sophisticated and substantial software solutions in the future

Bibliography

1. MongoDB Documentation
   MongoDB Manual – The Official MongoDB Guide URL: https://www.mongodb.com/docs/manual/
   Accessed: July 2025
2. Express.js Documentation
   Express – Node.js web application framework URL: https://expressjs.com/
   Accessed: July 2025
3. React.js Official Docs
   React – A JavaScript library for building user interfaces
   URL: https://reactjs.org/docs/getting-started.html
   Accessed: July 2025
4. Postman Learning Centre
   Using Postman for API Testing URL: https://learning.postman.com/
   Accessed: July 2025
5. GeeksforGeeks
   MERN Stack Tutorial and Full-Stack Web Development Articles
   URL: https://www.geeksforgeeks.org/mern-stack/
   Accessed: July 2025
6. W3Schools
   JavaScript, HTML, Node.js, and MongoDB Tutorials
   URL: https://www.w3schools.com/
   Accessed: July 2025
7. freeCodeCamp
   Full Stack Development Guides and YouTube Tutorials
   URL: https://www.freecodecamp.org/ Accessed: July 2025